TenderAI

How the application works, and how it meets the assignment

Task: AI agent for analysing Kazakhstan public procurement (ТЗ «AI-агент анализа государственных закупок РК»)
Scope: 27 customer organisations (BINs from the assignment), 2024–2026, goszakup.gov.kz OWS v3
Stack: Python, PostgreSQL 16, FastAPI, Ollama + Qwen3.5 9B on a local GPU, Docker / Kubernetes
Data as of: 2026-10-13 14:02 UTC · **Report date**: 18 October 2026

86.4% fact accuracy, 23-question held-out set (target ≥ 85%)	**97.3%** fact accuracy, 54-question corpus	**3 h** incremental sync interval (target: lag ≤ 24 h)	**78 678** priced observations behind the answers

Contents

1. Summary
2. Requirements compliance
3. Architecture
4. Data scope
5. Ingestion and freshness
6. Storage schema
7. ETL and data quality
8. Analytics and metrics
9. The AI agent
10. Example answers
11. Evaluation
12. API, web chat, deployment, security
13. Risks and limitations
14. Running the system

1. Summary

TenderAI answers questions about public procurement in Russian or Kazakh. It loads plans, announcements, lots, contracts and organisations for the 27 customers named in the assignment from the OWS v3 API, cleans them into an analytical warehouse, and answers through an agent that combines deterministic SQL/Python tools with a local language model.

> **Core design rule: the language model never calculates.** Every figure comes from SQL or Python. The model only classifies the question and writes a 1–3 sentence verdict; every number, BIN and date in that verdict is checked against the tool output, and so is the role each number plays. Five of the six answer sections are rendered directly from the tool output.

What a user gets for every question:

- A **verdict** of 1–3 sentences with the key figures.
- The **parameters** used: period, BINs, filters, how the item and customers were recognised, N.
- The **analytics**: averages, medians, baselines, deviations, comparisons.
- The **method** in one paragraph.
- **Confidence and limitations**: a confidence level, data-quality exclusion rates, applicable caveats.
- **Top-K** purchases, contracts or plan lines with direct goszakup.gov.kz links.

On a 23-question held-out set the agent was not tuned on, it extracts facts with **86.4%** accuracy (171 of 198, three runs); on the 54-question development corpus, 97.3%. The assignment requires at least 85%. Answers take about 4–6 seconds on an 8 GB laptop GPU.

2. Requirements compliance

Each requirement of the assignment (T3), how the application meets it, and where to find it.

Assignment requirement	How it is met	Where
Data: OWS v3, 27 BINs, 2024–2026	GraphQL + REST ingest for exactly these BINs and years; related plan lines and announcements outside the filter are fetched by ID	tenderai/config.py, ingest/
Storage schema (PostgreSQL/…)	PostgreSQL 16 with raw / core / mart / ops schemas, 10 migrations	db/sql, §6
ETL: units, junk in lot names	Unit normalisation to base units with factors; lot and plan names cleaned; DQ rules	etl/units.py, lot_names.py, dq.sql
Analytical layer (marts for the LLM)	price_obs, price_stats, price_deviation, volume_yearly, dq_summary	etl/marts.sql, §6
Questions in KZ / RU	Language detected per question; answers and section titles in the same language	agent/router.py, i18n.py
Question types: search, comparison, analytics, anomalies, fairness of price and quantity	Six intents and seven tools; fairness of a lot covers both its price (Fair Price) and its quantity (customer's planned volume vs earlier years)	agent/tools.py, §9
Factual analysis, explainable and reproducible	Deterministic tools; answers rebuilt from stored data; every figure traceable to an object ID	§9
Fact accuracy ≥ 85% (numbers, dates, BINs)	86.4% on the 23-question held-out set (unseen questions); 97.3% on the 54-question corpus, 100.0% on the golden set	eval/, §11
Incremental update, lag ≤ 24 h	Sync every 3 h (worker or K8s CronJob): changes by indexDate + deletions from the journal; /health reports lag and the SLA flag	ingest/incremental.py, §5
Mandatory: anomalies > 30% from the weighted average by ENSTRU	Leave-one-out weighted average in constant prices, threshold from the question, robust z confirmation	§8.2

Assignment requirement	How it is met	Where
Mandatory: fairness of a lot vs other agencies in the same city	Fair Price with same city → region → national → code prefix comparison levels, other customers only	§8.1
Mandatory: atypical growth of quantity vs previous years	Ratio to the median of earlier years + robust z; confirmed tier; Isolation Forest as a secondary score	§8.3
Methods: median, IQR, MAD, percentiles; unsupervised ML; rules	All used: medians/IQR/MAD/quartiles, robust z, Isolation Forest, rule-based routing and thresholds	analytics/
Fair Price: comparable ENSTRU codes, regional coefficient, inflation and seasonality	Formal 6-step definition: comparable set, official CPI by region and category, seasonal index, shrunk regional coefficient, median and Q1–Q3 band, confidence	§8.1
Explainability: IDs, N compared, objects analysed, method, confidence and limitations	Mandatory in every answer, including comparisons and statistics (Top-K examples, aggregate confidence)	§9.5
Only data from the API and warehouse; no generic phrases or guessing; numbers required	Numbers come only from tools; validator rejects unsupported or misused figures; questions without data get a 'no data' answer with the reason, never invented numbers	§9.4
Answer structure (6 sections)	Verdict · parameters · analytics · metric · confidence and limitations · Top-K	agent/composer.py
Data coverage: announcements, plans, contracts, lots, goods/works/services, customers, suppliers, statuses, dictionaries	All loaded; subject type (goods/works/services) on plans, announcements and contracts; 12 REST dictionaries	§4
Architecture API → storage, schema, analytics, agent logic; LLM uses tools	§3, §6, §8, §9	docs/
Non-functional: data quality control	DQ rules with error/warning severity; rates in every answer	§7
Non-functional: API token protection	Token only in env / secrets, SecretStr, request logging silenced	§12
Non-functional: on-prem Docker / Kubernetes	docker-compose (5 services) and K8s manifests; local LLM, no internet needed at query time	deploy/, §12
Deliverables: architecture, schema, analytics, examples, risks	docs/architecture.md, schema.md, analytics.md, examples.md, risks.md, data_scope.md, and this report	docs/

3. Architecture

Four layers: external sources, ingestion, the PostgreSQL warehouse, and the agent with its API. The agent reads only the cleaned core and mart schemas through a read-only database role; the local LLM runs next to it.

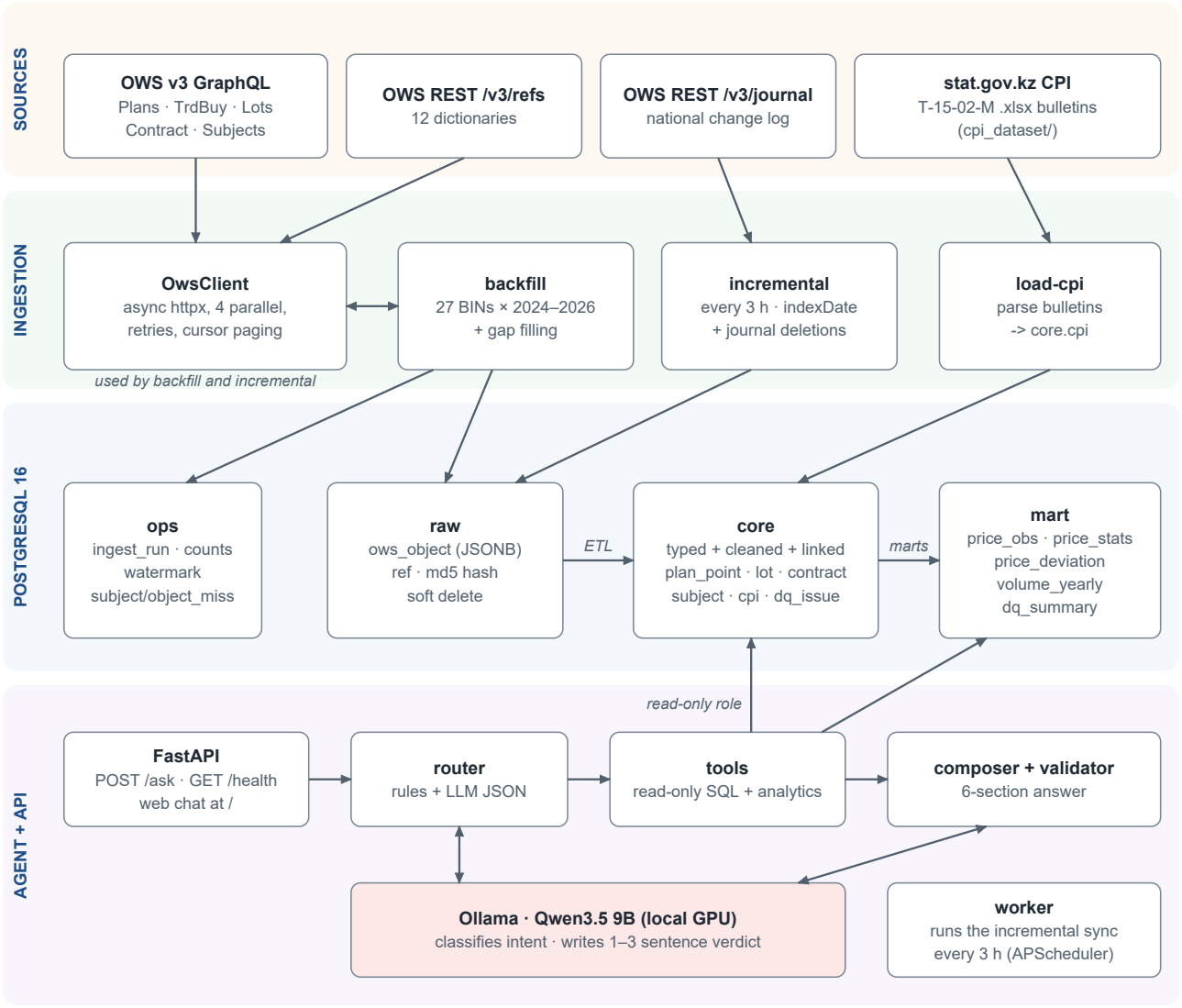


Figure 1. Components and data flow.

Component	Responsibility	Code
OWS client	Async HTTP, bearer token, 4 parallel requests, retries with backoff, cursor paging, errors in HTTP-200 bodies detected	ows/client.py
Backfill / incremental	Full load; then every 3 h: changes since the watermark (2 h overlap) and deletions from the journal; follow-up fetches; ETL rebuild	ingest/
ETL	raw → core → marts in one transaction (~1 min)	etl/
Analytics	Fair Price, price index and CPI, robust statistics, volume anomalies	analytics/
Router, tools, validator, composer	Question understanding, deterministic tools, fact and claim checks, six-section answer	agent/
API and web chat	POST /ask, GET /health, chat page	api/
Worker	Incremental sync every 3 h	worker/

4. Data scope

The OWS v3 entities used and why. GraphQL is the main path: it carries full lot data and supports filters by BIN, year and indexDate; REST is used for dictionaries and the change journal.

Entity	Loaded by	Used for
Plans (plan lines)	customer BIN + plan year 2024–2026, plus referenced lines by ID	ENSTRU code, unit, planned price and quantity, delivery KATO, goods/works/services type; volume anomalies
Announcements (TrdBuy)	organiser BIN + publish date, plus referenced by ID	date, trade method, status, link
Lots	customer BIN	announced price and quantity; link to plan line and announcement
Contracts + line items	customer BIN + sign date	actual unit prices without VAT, supplier, status, amendments
Subjects	the 27 customers + every supplier on a contract	names, addresses (fallback location), roles
Dictionaries (12, REST)	/v3/refs/*	units, KATO, trade methods, all statuses, subject types, finance sources
Journal (REST)	/v3/journal	deletions for the incremental sync

Current warehouse: 152 246 plan line versions, 65 835 announcements, 273 110 lots (117 106 in scope), 36 831 contract versions with 105 324 line items, 8 849 organisations; 537 986 raw API objects in total. Latest plan lines by subject type — Товар: 136 519, Услуга: 13 024, Работа: 2 258.

5. Ingestion and freshness

- **Backfill** (about 25 min): dictionaries; per-BIN loads of plans, announcements, lots and contracts, with the API's reported total stored next to the rows fetched; plan lines and announcements referenced from outside the filter; customers and all suppliers.
- **Incremental** (every 3 h): the same per-BIN queries with indexDate ≥ watermark − 2 h; deletions from the national journal (server-filtered to action=D; objects re-indexed inside the window are kept); newly referenced objects; then the ETL; then the watermark moves.
- **Idempotent storage**: raw payloads keyed by (entity, id) with a content hash, so re-runs are safe; deletions are soft.
- **Freshness monitoring**: GET /health reports the data timestamp, lag in hours and whether the 24-hour target is met.
- **Journal volume**: the national journal normally holds about 105,000 changes a day, but 4.6 million on 12 October 2026 when the portal re-indexed plans in bulk. Reading it in full would have taken about 2.5 hours, so the sync asks the server for deletions only (action=D: 746 pages that day) and never deletes an object it has just fetched live.

Latest runs: #12 incremental ok at 2026-10-13 14:02; #11 incremental failed at 2026-10-13 11:53; #10 incremental failed at 2026-10-13 11:13; #9 incremental ok at 2026-10-12 10:32. Data timestamp after the latest sync: **2026-10-13 14:02 UTC**.

The latest successful sync covered two days of the journal (1,644 deletion pages) and took about an hour, well within the 3-hour cycle; it soft-deleted 523 objects (lot: 503, plan_point: 20) and rebuilt the warehouse in about a minute. Run #10 (the old full-journal scan) was stopped to make that fix and run #11 was interrupted by a machine shutdown; both are recorded as failed with the reason, and nothing from them was committed.

6. Storage schema

Schema	Purpose
raw	Exact API payloads (JSONB) and dictionaries; change hash; soft-delete flag
core	Typed, cleaned and linked entities: plan_point, announcement, lot, contract, contract_unit, subject, dictionaries, cpi, dq_issue, target_customer
mart	What the agent reads: price_obs, price_stats, price_deviation, volume_yearly, dq_summary
ops	Run log, per-BIN reconciliation, watermarks, lists of objects the API no longer returns

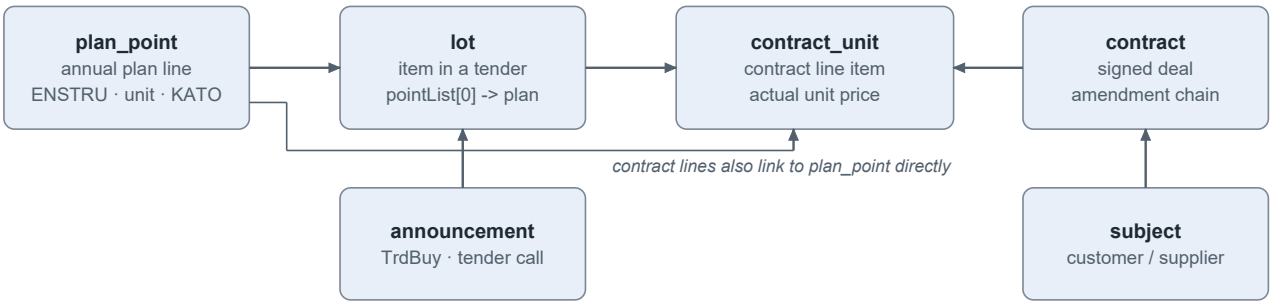


Figure 2. How procurement records link: plan line → lot → contract line.

Mart	Grain	Rows
price_obs	one priced observation: a contract line, or a lot with no contract	78 678
price_stats	ENSTRU × unit × period	33 880
price_deviation	observation + leave-one-out peer average + robust z	66 341
volume_yearly	customer × ENSTRU × unit × year (planned)	39 271

The agent connects as `tenderai_agent`: read-only, core and mart only, 15-second statement timeout. The LLM never writes SQL; tools use fixed templates with bound parameters.

7. ETL and data quality

Step	What happens
Units	Each MKEI unit maps to a base unit and factor (g/kg/t → kg, ml/L/m³ → L, thousand pcs → pcs); packaging units (pack, set) stay separate and are compared only with themselves
Names	Lot and plan names are cleaned of numbering, quotes, specification tails and noise into name_clean
Versions	Only the latest plan version and the final contract of each amendment chain are used
Location	Delivery KATO: lot → plan → announcement → customer address, with the source recorded
Prices	Without VAT, per base unit; a contract line replaces the lot it came from
CPI	Official stat.gov.kz bulletins (Jan 2024 – Sep 2026), by region and category, loaded into core.cpi
DQ rules	Errors (non-positive price or quantity, missing unit or code) are excluded from statistics; warnings are kept and lower confidence. 236 601 issues recorded

Entity	Rule	Severity	Count
plan_point	delivery_kato_inferred_or_missing	warning	117 199
plan_point	year_out_of_scope	warning	71 752
lot	no_plan_point_link	warning	34 335
contract_unit	missing_unit_or_enstru	error	4 964
contract_unit	unit_inferred_from_enstru	warning	4 380

Entity	Rule	Severity	Count
lot	lot_price_differs_from_plan	warning	3 080
contract_unit	total_ne_price_x_quantity	warning	754
contract	contract_exceeds_lot_amount	warning	55

8. Analytics and metrics

8.1 Fair Price (formal definition)

Step	Rule
1. Comparable set	Same ENSTRU code and base unit, other customers, within ±18 months. The first level with N ≥ 5 is used: same delivery city → same region → national → coarser code prefix.
2. Time	$p' = p_i \cdot I(t) / I(t_i) \cdot S(m_t) / S(m_i)$. I is the official CPI of the comparable's region and the item's category; S is a seasonal index (only with N ≥ 30 over ≥ 6 months).
3. Region	$K(r) = w \cdot \text{median}(p' \text{ in } r) / \text{median}(p') + (1 - w)$, $w = n_r / (n_r + 10)$; prices are moved to the target region.
4. Fair price	Median of adjusted prices; fair band = [Q1, Q3]; weighted mean reported alongside.
5. Verdict	Overpriced if deviation > +30% or robust z > 3; underpriced (dumping risk) if < −30% or z < −3; fair if inside the band or within ±10%; otherwise elevated / below market.
6. Confidence	$0.35 \cdot \min(1, \ln N / \ln 30) + 0.25 / (1 + \text{IQR/median}) + 0.20 \cdot \text{level} + 0.10 \cdot (1 - \text{DQ}) + 0.10 \cdot$ region known.

Quantity of the lot. The same answer judges the quantity: the customer's planned volume of that code and unit in the lot's year against the median of its earlier years; above 1.5× is reported as atypical.

8.2 Price anomalies

Every observation is converted to constant prices with the official CPI. Its baseline is the quantity-weighted average of the **other** purchases of the same code and unit (leave-one-out, at least 3 peers), so an outlier cannot hide in its own average. Deviation above the threshold from the question (30% by default) flags it; a robust z of the log price above 3 confirms it. Prices more than 21× the peers are set aside as likely unit errors. Services priced per '1 service' are excluded. 'Below market' questions use the same method in the other direction.

8.3 Volume anomalies

Planned quantity per customer × code × unit × year against the median of earlier years. Flagged above 1.5× (and robust z > 3 when two earlier years exist); confirmed when the growth is statistically extreme and the extra planned spend is at least 1 million KZT. An Isolation Forest score is shown as a secondary signal only.

8.4 Confidence of other answers

Search, comparisons, statistics and anomaly answers carry an aggregate confidence: $0.6 \cdot \min(1, \ln N / \ln 30) + 0.4 \cdot (1 - \text{DQ exclusion rate})$; high ≥ 0.70, medium ≥ 0.45, low otherwise, and always low under 5 objects.

9. The AI agent

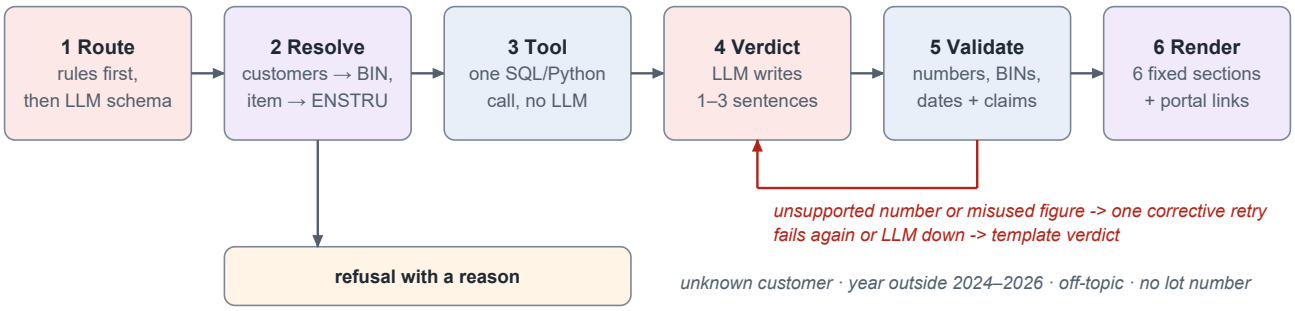


Figure 3. Lifecycle of one question. Red steps call the local LLM; everything else is deterministic.

9.1 Understanding the question

- **Rules first**: language, BINs, ENSTRU codes, lot numbers, years (2025 vs 2024 means 2025; years outside 2024–2026 are refused), thresholds, top-K, a unit named in the question, 'below market' direction.
- **Then the LLM** fills a JSON schema: intent, item, customer names, year, threshold, top-K. Rule facts always win; ranking questions are comparisons; a fairness intent without a lot falls back to the rules.

9.2 Resolving names

- **Customers**: several names are split; each is matched word by word against the 27 customers after collapsing official forms (e.g. 'Управления делами Президента' → 'УДП'); one best match per name. An unmatched name is **refused**, never widened to all customers.
- **Items**: matched to ENSTRU codes in tiers — exact name, phrase, all words, trigram similarity — preferring the most-purchased code; search covers every code sharing the item's name.

9.3 Tools

Intent	Tool	Output
search	search_procurements	count, total, Top-K purchases
analytics	price_stats	weighted average, median, Q1–Q3, min/max per year + examples
compare	compare_customers / compare_years	per-customer or per-year table + examples
price_anomaly	price_anomalies	flagged, confirmed, estimated overpayment, Top-K deviations
fair_price	fair_price_for_lot	Fair Price, band, deviation, verdict, confidence, quantity check
volume_anomaly	volume_anomalies	series compared, flagged, confirmed, extra spend, Top-K series

9.4 Verdict and validation

The LLM receives the question, a draft and the facts as JSON, and writes 1–3 sentences (temperature 0, fixed seed, thinking off). The validator then:

- matches every number, BIN and date (Russian and Kazakh formats, '12,5 млн') to the facts;
- checks the role of numbers: the compared total cannot be presented as the flagged count; a price outside Q1–Q3 cannot be called inside it;
- corrects 'ИНН' on an organisation's number to 'БИН'.

On a problem the model gets one corrective retry naming it; if that fails too, a template verdict is used.

9.5 Answer format

Assignment section	Answer section	Source
Вердикт / краткий вывод	Вердикт (1–3 sentences)	LLM, validated; or template
Параметры поиска / использованные данные	Параметры поиска и использованные данные	tool
Аналитика / сравнение	Аналитика и сравнение	tool
Метрика оценки	Метрика оценки	tool
Уверенность / ограничения	Уверенность и ограничения	tool
Детализация / примеры Top-K	Детализация (Top-K) with links	tool

10. Example answers

Real answers from the current data, as the agent returns them (portal links shortened to their labels; long Top-K lists trimmed to 5).

Mandatory type: price anomalies

Question: Найди закупки с отклонением цены > 30% от средневзвешенной по данному ЕНСТРУ, топ-5 · intent price_anomaly · verdict: llm · 19.3 s

Вердикт
Из 58 009 сопоставимых закупок 1 589 позиций (2,7%) подтверждены как резко выбивающиеся с отклонением цены более 30% и общей оценкой возможной переплаты в 1 227 856 334 ₸. Топ-5 таких аномалий включает закупку топлива дизельного (+58,8%), салфеток (+302,8%) и автомобиля (+34,1%).

Параметры поиска и использованные данные
• **Период:** 2024–2026
• **Заказчики (БИН):** все 27 заказчиков из перечня
• **Фильтры:** threshold = 0,30; direction = over; include_services = False
• **Объектов в анализе:** 58009
• *Источник: API OWS v3 → хранилище TenderAI*

Аналитика и сравнение
• Сопоставимых позиций (≥3 аналога): 58009 в 4039 группах ЕНС ТРУ × ед.
• Отклонение более порога (правило): 18760 (32,3%), оценка переплаты 8 719 727 945 ₸
• Подтверждено робастной проверкой (z > 3): **1589** (2,7%), оценка переплаты **1 227 856 334 ₸**
• Вероятные ошибки ед. изм./количества (> 21× аналогов, исключены из оценки): 1135

Метрика оценки
отклонение цены за единицу (без НДС, в базовой ед. изм., в сопоставимых ценах по ИПЦ) от средневзвешенной по количеству цены остальных закупок того же ЕНС ТРУ и единицы (leave-one-out, группы с ≥3 аналогами); подтверждение — робастный z-score логарифма цены в группе (медиана/MAD) > 3

Уверенность и ограничения
• **Уверенность:** высокая (0,98)
• **Исключено по качеству данных:** contract_unit_excluded_rate = 4,7%
• средневзвешенная считается в сопоставимых ценах (официальный ИПЦ stat.gov.kz по региону и категории); средняя аналогов показана в ценах месяца каждой закупки
• разброс цен внутри одного кода ЕНС ТРУ велик (разные фасовки/характеристики), поэтому правило «> порога» дополнено робастной проверкой: подтверждённые аномалии — robust z (лог-цена) > 3
• цена выше аналогов более чем в 21 раз считается вероятной ошибкой единицы/количества (лот «одной суммой»), такие позиции показаны отдельно и не входят в оценку переплаты
• услуги и работы (ед. изм. «услуга»/«работа») исключены: цена за единицу для них не сопоставима

Детализация (Top-5)
1. contract_unit:44502273 — ✓ топливо дизельное — 400,78 ₸ vs 252,34 ₸ (**+58,8%**, z=8,1, аналогов 99), заказчик 000740001307, 01.2024
2. contract_unit:55804532 — ✓ салфетка — 4 199 ₸ vs 1 042,57 ₸ (**+302,8%**, z=3,7, аналогов 5), заказчик 990740002243, 02.2026
3. contract_unit:49198394 — ✓ автомобиль — 84 800 000 ₸ vs 63 258 349,02 ₸ (**+34,1%**, z=16,5, аналогов 3), заказчик 940740000911, 12.2024
4. contract_unit:52330744 — ✓ устройство многофункциональное — 46 430 188 ₸ vs 4 907 257,22 ₸ (**+846,2%**, z=8,5, аналогов 5), заказчик 981140001551, 06.2025
5. contract_unit:49473094 — ✓ электрокардиостимулятор — 5 000 000 ₸ vs 873 645,53 ₸ (**+472,3%**, z=4,7, аналогов 20), заказчик 990740002243, 12.2024

Mandatory type: fairness of a lot

Question: Оцени адекватность цены лота №74823661-ЗЦП1 относительно аналогичных контрактов других ведомств в идентичном городе поставки · intent fair_price · verdict: llm · 9.8 s

Вердикт
Цена лота 74823661-ЗЦП1 составляет 866,07 ₸ за кг, что на 2,4% выше справедливой цены в 846,08 ₸ при сравнении с 33 аналогами из одного региона. Стоимость лота находится в пределах разброса цен аналогов от 752,07 ₸ до 1 029,20 ₸, что подтверждает её адекватность относительно рыночного диапазона других ведомств.

Параметры поиска и использованные данные
• **Период:** 2024–2026
• **Заказчики (БИН):** все 27 заказчиков из перечня
• **Фильтры:** lot_ref = 74823661-ЗЦП1
• **Объектов в анализе:** 33
• *Источник: API OWS v3 → хранилище TenderAI*

Аналитика и сравнение
• Лот: 74823661-ЗЦП1 — Огурец (ЕНС ТРУ `011332.100.000000`, kg), заказчик 990740002243
• цена лота (договор ещё не заключён): **866,07 ₸** / kg
• Fair Price: **846,08 ₸**, диапазон Q1–Q3: 752,07 – 1 029,20 ₸; средневзвешенная: 844,40 ₸
• Отклонение: **2,4%**, robust z = 0,10
• Уровень сравнения: `same_city`, N = 33
• Инфляция: официальный ИПЦ stat.gov.kz (продовольственные товары, по региону каждого аналога), коэффициенты 0,900–1,106; региональный коэффициент: не применялся — все аналоги из одного региона; сезонность: применена
• Количество (план заказчика по этому ЕНС ТРУ): 2025 — 9 764 kg vs 2024: 8 987 (×1,09) — в пределах обычного

Метрика оценки
Fair Price: медиана цен аналогов (тот же ЕНС ТРУ и ед. изм., другие заказчики, ±18 мес.) с поправками на инфляцию/сезонность и региональный коэффициент; полоса справедливости = Q1–Q3

Уверенность и ограничения
• **Уверенность:** высокая (0,94)
• **Исключено по качеству данных:** lot_excluded_rate = 0,0%
• место поставки лота определено косвенно (источник: announcement)

Детализация (Top-10)
1. contract_unit:55026799 — аналог: 900 ₸, заказчик 020440003656, 01.2026, KATO 711210000
2. contract_unit:56901173 — аналог: 950 ₸, заказчик 971040001050, 05.2026, KATO 711210000
3. contract_unit:53801506 — аналог: 906,50 ₸, заказчик 971040001050, 11.2025, KATO 711210000
4. contract_unit:52830422 — аналог: 810 ₸, заказчик 020440003656, 07.2025, KATO 711210000
5. contract_unit:52830423 — аналог: 810 ₸, заказчик 020440003656, 07.2025, KATO 711210000

Mandatory type: volume growth

Question: Выяви нетипичное завышение количества ТРУ по сравнению с предыдущими годами, топ-5 · intent volume_anomaly · verdict: llm · 9.9 s

Вердикт
Из 12 898 рядов сравнения рост планового количества ТРУ более чем в 1,5 раза выявлен в 2 768 случаях (21,5%), из которых статистически подтверждены как нетипичные и существенные 482 случая с дополнительными плановыми расходами 3 481 028 915 ₸.

Параметры поиска и использованные данные
• **Период:** 2024–2026
• **Заказчики (БИН):** все 27 заказчиков из перечня
• **Фильтры:** ratio_threshold = 1,50; include_services = False
• **Объектов в анализе:** 12898
• *Источник: API OWS v3 → хранилище TenderAI*

Аналитика и сравнение
• Рядов сравнено: 12898; рост > 1,5× (правило): 2768 (21,5%)
• Подтверждено (статистически нетипично и ≥ 1 млн ₸): **482** (3,7%), доп. плановые расходы **3 481 028 915 ₸**

Метрика оценки
годовой объём по пунктам плана (последние версии) на заказчика × ЕНС ТРУ × ед. изм.; флаг: объём / медиана предыдущих лет > 1.5 и robust z (лог-объём) > 3 при ≥2 предыдущих годах; подтверждено: z > 3 (или рост ≥ 3× при одном предыдущем годе) и дополнительные плановые расходы ≥ 1 млн ₸; Isolation Forest — дополнительный балл

Уверенность и ограничения
• **Уверенность:** высокая (1)
• **Исключено по качеству данных:** plan_point_excluded_rate = 0,0%
• история ограничена 2024–2026 гг.: для 2025 г. база — один предыдущий год
• сравниваются плановые объёмы; фактическое исполнение может отличаться
• услуги и работы исключены: их «количество» — число договоров, а не объём

Детализация (Top-5)
1. 262021.300.000003 — ✓ массив дисковый (pcs), заказчик 210240033968: 2025 — 3 vs 2024: 1 (**×3**, z=—, IF=0,618)
2. 101111.400.000008 — ✓ говядина (kg), заказчик 990740002243: 2025 — 59 944 vs 2024: 18 000 (**×3,33**, z=—, IF=0,401)
3. 205952.100.000293 — ✓ набор реагентов (u778), заказчик 990740002243: 2026 — 440 vs 2024: 68, 2025: 33 (**×8,71**, z=4,16, IF=0,666)
4. 192026.510.000000 — ✓ топливо дизельное (L), заказчик 971040001050: 2025 — 303 803 vs 2024: 17 500 (**×17,36**, z=—, IF=0,558)
5. 205952.100.000180 — ✓ набор реагентов (u704), заказчик 990740002243: 2025 — 267 vs 2024: 17 (**×15,71**, z=—, IF=0,671)

Kazakh question

Question: 2025 жылы жазғы дизель отынының орташа бағасы қанша? · intent analytics · verdict: llm · 5.9 s

Қорытынды

2025 жылы жазғы дизель отынының орташа бағасы 36 сатып алу бойынша 286,45 теңге құрады. Бұл деректерді тексеру үшін медиана (285,67 теңге) және Q1–Q3 диапазондарына (263,39–297,77 теңге) сүйенген дұрыс.

Іздеу параметрлері және пайдаланылған деректер

• **Кезең:** 2025
• **Тапсырыс берушілер (БСН):** тізімдегі барлық 27 тапсырыс беруші
• **Сүзгілер:** enstru_code = 192026.510.000000, 192026.510.000001, 192026.520.000000, 192026.510.000002; base_unit = L
• **Сатып алу нысаны анықталды:** топливо дизельное; нұсқалар: летнее (`192026.510.000000`), зимнее (`192026.510.000001`), межсезонное (`192026.520.000000`), арктическое (`192026.510.000002`)
• **Талдаудағы объектілер:** 51
• *Дереккөз: OWS v3 API → TenderAI қоймасы*

Талдау және салыстыру

Нұсқа (ЕНС ТРУ)	Кезең	N	Тапсырыс берушілер	Орт. өлш.	Медиана	Q1	Q3	Мин	Макс
летнее (`192026.510.000000`, L)	2024	32	12	293,44	263,19	259,80	263,39	245,54	400,78
летнее (`192026.510.000000`, L)	2025	36	12	286,45	285,67	263,39	297,77	257,15	348
летнее (`192026.510.000000`, L)	2026	32	13	290,86	298,31	292,89	303,67	278,45	324
летнее (`192026.510.000000`, L)	all	100	15	290,24	285,67	263,39	300,43	245,54	400,78
зимнее (`192026.510.000001`, L)	2024	11	6	610,91	550	508,40	660,71	262,90	669,64
зимнее (`192026.510.000001`, L)	2025	9	5	463,80	515,70	413,30	616,07	263,38	616,07
зимнее (`192026.510.000001`, L)	2026	15	5	573,43	594,83	594,83	594,83	510	616,07
зимнее (`192026.510.000001`, L)	all	35	7	562,50	594,83	521,23	602,56	262,90	669,64
межсезонное (`192026.520.000000`, L)	2024	1	1	263,39	263,39	263,39	263,39	263,39	263,39
межсезонное (`192026.520.000000`, L)	2025	2	1	279,80	274,55	268,97	280,13	263,39	285,71
межсезонное (`192026.520.000000`, L)	2026	2	2	434,04	419,54	354,27	484,80	289	550,07
межсезонное (`192026.520.000000`, L)	all	5	2	300,26	285,71	263,39	289	263,39	550,07
арктическое (`192026.510.000002`, L)	2025	4	1	510,18	522,32	514,28	525,67	490,17	535,71

арктическое (192026.510.000002, L)	all	4	1	510,18	522,32	514,28	525,67	490,17	535,71

Бағалау әдісі
ЕНС ТРУ және жыл бойынша бірлік бағасының (ҚҚС-сыз) сипаттамалық статистикасы
Сенімділік және шектеулер
• **Сенімділік:** жоғары (0,98)
• **Деректер сапасы бойынша алынып тасталды:** contract_unit_excluded_rate = 4,7%; lot_excluded_rate = 0,0%
• бір ЕНС ТРУ коды әртүрлі қаптамалар мен сипаттамаларды біріктіруі мүмкін — тек орташаға емес, медиана мен Q1–Q3 аралығына сүйеніңіз
• тауар нұсқалары (әртүрлі ЕНС ТРУ кодтары) бөлек көрсетілген: бұл әртүрлі тауарлар, олардың бағалары бір орташаға біріктірілмейді
Толығырақ (Top-10)
1. contract_unit:51241529 — топливо дизельное — 127 466 985 ₸ (366 284 × 348 ₸), тапсырыс беруші 940740000911, 03.2025
2. contract_unit:54386395 — топливо дизельное — 123 687 944 ₸ (469 600 × 263,39 ₸), тапсырыс беруші 000740001307, 12.2025
3. contract_unit:53438909 — топливо дизельное — 96 643 800 ₸ (330 000 × 292,86 ₸), тапсырыс беруші 990740002243, 09.2025
4. contract_unit:50487799 — топливо дизельное — 86 918 700 ₸ (330 000 × 263,39 ₸), тапсырыс беруші 990740002243, 02.2025
5. contract_unit:52135177 — топливо дизельное — 55 350 000 ₸ (200 000 × 276,75 ₸), тапсырыс беруші 990740002243, 05.2025

Customer outside the 27

Question: Покажи закупки акимата города Алматы за 2025 год · intent search · verdict: template · 1.9 s

Вердикт
Данных, удовлетворяющих запросу, в хранилище нет — ответ без цифр не даётся. Заказчик «акимат города Алматы» не найден среди 27 заказчиков хранилища.
Параметры поиска и использованные данные
• **Запрошено (поиск не выполнялся):** intent = search; customer = акимат города Алматы; years = 2025
• **Объектов в анализе:** 0
• *Источник: API OWS v3 → хранилище TenderAI*
Уверенность и ограничения
• заказчик «акимат города Алматы» не найден среди 27 заказчиков хранилища

11. Evaluation

Four question sets check answers automatically. Each case lists the facts (BINs, codes, numbers) a correct answer must contain; numbers are parsed from Russian/Kazakh formatting with 0.5% tolerance. Facts the question itself contains (a BIN or lot number the user typed) are not scored, and small numbers must appear outside the Top-K list. Fair Price verdicts and refusals ('no data', out of scope) are scored as separate checks. The held-out set was written after the agent was finished, on customers, items and wordings used nowhere else, and the agent is not changed to pass it; eval/check_leakage.py (also a unit test) verifies that no evaluation question is among the LLM's few-shot examples and no evaluation identifier or figure appears in the code.

Set	Cases	Fact accuracy	Verdict / refusal checks	Intent accuracy
Held-out (eval/heldout.jsonl): fresh questions, not tuned on	23	86.4% (171/198)	100.0% (21/21)	95.7%
Corpus (eval/corpus.jsonl): hand-picked questions incl. traps; used during development	54	97.3% (438/450)	100.0% (39/39)	96.3%
Golden set (eval/golden.jsonl): templated questions from SQL	40	100.0% (89/89)	—	100.0%
Held-out, without the LLM (rules fallback)	23	54.5% (36/66)	85.7% (6/7)	82.6%
Corpus, without the LLM	54	61.3% (92/150)	100.0% (13/13)	94.4%

Set	Cases	Fact accuracy	Verdict / refusal checks	Intent accuracy
Golden set, without the LLM	40	100.0% (89/89)	—	100.0%

Product variants. One product is often split into codes that differ only in the plan description (petrol АИ-92 / АИ-95, summer / winter diesel, A4 / A3 paper, battery types). Item resolution reads those descriptions (mart.enstru_variant): a named variant ('бензин АИ-95') selects its code, a bare product name ('бензин') selects every variant; spending and anomalies cover all of them, and price statistics are shown per variant, never pooled. Before this change the held-out set scored 71.0% and the corpus 80.7%. Remaining misses: Kazakh item names the LLM transliterates instead of translating ('пияз' became 'пьяз'), a customer name the LLM also copies into the item field ('Бүркіт'; varies between sessions), and abbreviations ('дизтопливо').

Without the LLM the agent still routes questions and recognises customers by name in the question text, and answers with template verdicts. It cannot turn free-text item names into product codes, which is why the corpus, whose questions name items in words, scores lower in that mode; questions that give a code, a lot number or a BIN are unaffected.

Stability

The LLM is not fully deterministic, so a single pass can hide answers that only go wrong some of the time. Every question is therefore also asked 3 times, and the paraphrase set (eval/paraphrases.jsonl) asks each mandatory question type (and one analytics question) 5–7 different ways in Russian and Kazakh; every wording in a group must give the same facts.

Set	Questions × runs	Fact accuracy	Worst run	Same answer in every run
Held-out	23 × 3	86.4%	86.4%	100.0%
Corpus	54 × 3	97.3%	97.3%	100.0%
Paraphrases (4 groups)	24 × 3	94.8%	94.8%	100.0%

The first paraphrase run found a Kazakh wording ('ТЖҚ саны күрт өскен…') wrongly refused as off-topic; the check now applies only to unclassified questions. A user-reported answer that analysed 0 purchases, because the model took 'ЕНСТРУ' as the item, led to two safeguards: classifier words are never items, and an item matching no product code and no purchase name is reported as unrecognised instead of 'no data'.

The corpus was written after studying the data: customers named by name (not BIN), everyday items, lots with a known fair / overpriced / underpriced verdict, and edge cases (unknown customer, year outside the period, off-topic, missing lot number, item names that fuzzy matching gets wrong). Its reference answers are computed with independent SQL, and volume anomalies with an independent implementation of the rule.

Stage	Fact accuracy	Main problems
First run with the LLM	66.9%	unknown customers widened to all 27; 'бензин' matched a medicine; two customers in one question; years; 'below market'
After matching and routing fixes	99.3%	unit named in the question ignored; numbers used in the wrong role
After units and claim checks	100.0%	—
Re-run on refreshed data (13 Oct), corpus rebuilt	96.0%	a code represented by a stray 'приобретение …' name; the LLM's routing of a two-year comparison varied between runs
After name cleaning and a year-comparison rule	100.0%	—
Review (14 Oct): references corrected, scoring stricter, few-shot examples replaced	80.7%	variant codes (АИ-92 / АИ-95, summer / winter diesel, A4, laundry soap)
Product variants told apart by plan description	97.3%	a Kazakh item name the LLM did not translate

Caveat: the fixes were prompted by this corpus, so its score is optimistic. A later review found that several of its reference answers treated one variant code as the whole product (petrol = АИ-92 only, diesel = summer only, 'A4 paper' including A3 and A5, 'soap' = laundry soap only) and that the router's few-shot examples contained evaluation questions. Corrected references, stricter scoring and new examples gave 80.7% on the corpus; telling product variants apart raised it to 97.3%. The held-out set above is the honest measure.

12. API, web chat, deployment, security

Endpoint	Behaviour
GET /	Web chat: question box, examples, answers rendered as six labelled sections with links
POST /ask	Full structured answer: markdown, route, tool result, verdict source, validation, timings; requests are serialised because one local GPU serves the LLM
GET /health	Database, LLM availability, data timestamp, lag in hours, 24 h SLA flag, last ingest run

Deployment	Contents
Docker Compose	postgres 16, ollama (GPU), one-off migrate, api, worker — one non-root application image
Kubernetes	Postgres StatefulSet, Ollama Deployment with a GPU, migrate Job, API Deployment with readiness on /health, incremental CronJob every 3 h, secrets in a Secret

- The OWS token lives only in the environment or a secret, is a SecretStr in settings and never reaches logs.
- The agent's database role is read-only and limited to core and mart.
- The LLM runs locally with no internet, tools or SQL access; user-supplied text reaches it only inside a JSON facts block, and its output is validated.

13. Risks and limitations

Risk	Mitigation
Two ENSTRU formats, broad codes, wide price spread within a code	Exact then prefix matching; leave-one-out baselines; robust z confirmation; medians and IQR
≈70% of plan lines without a delivery location	Fallback chain with recorded source; stated in the answer
Planned vs executed volumes; short history (2024–2026)	Stated in every volume answer; z-test only with two earlier years
Only 27 customers, so peers are the other 26	N and comparison level shown; more BINs is a config change
CPI bulletins end in September 2026	Nearest month used; add new bulletins and re-run load-cpi
LLM wording errors	Fact and claim validation, retry, template fallback; deterministic sections
Name and item recognition	Tiered matching, refusal of unknown customers, resolved code shown to the user
One 8 GB GPU	Requests serialised; template answers when the LLM is unavailable

> A price 'anomaly' is a signal for review, not proof of a violation. The fair band reflects what the other 26 customers actually paid; it is not an external market benchmark.

14. Running the system

```
cp .env.example .env                          # OWS_TOKEN
docker compose -f deploy/docker-compose.yml up -d postgres ollama
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen3.5:9b
python -m tenderai.cli migrate
python -m tenderai.cli backfill               # ~25 min
python -m tenderai.cli load-cpi cpi_dataset
python -m tenderai.cli etl
python -m tenderai.cli serve                  # http://127.0.0.1:8010
python -m tenderai.cli worker                 # incremental sync every 3 h
python eval/run_eval.py eval/corpus.jsonl     # fact accuracy
```